

Code-Layout, Readability and Reusability

Date	29 June 2026
Team ID	N/A
Project Name	OptiCrop Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

The following table describes how code reusability is achieved in the OptiCrop project.

Module / Component	Reusable Code	Purpose	Benefits
Data Preprocessing	Reusable preprocessing functions for cleaning, validation, and feature preparation.	Ensures consistent data processing before prediction.	Reduces duplicate code and improves maintainability.
Machine Learning Model	Random Forest model saved as a .pkl file.	Can be reused without retraining for every prediction.	Improves performance and deployment efficiency.
Prediction Module	Common prediction function used across all scenarios.	Provides crop recommendation and suitability analysis.	Avoids repeated implementation and simplifies updates.
Flask Routes	Reusable Flask routes and templates.	Handles user requests and displays prediction results.	Improves modularity and easier UI maintenance.
Input Validation	Reusable validation methods.	Checks input ranges for N, P, K, pH, temperature, humidity, and rainfall.	Prevents invalid inputs and improves reliability.
Recommendation Engine	Reusable recommendation generation logic.	Produces farming suggestions based on prediction results.	Easy to extend with additional recommendations.
Visualization Module	Reusable plotting functions using Matplotlib and Seaborn.	Generates graphs and analysis reports.	Supports research and future enhancements.
Configuration	Centralized configuration and requirements.	Stores project settings and dependencies.	Simplifies deployment across different systems.

Assessment Area	Assessment	Remarks
Modularity	High	Separate preprocessing, ML, prediction, Flask, and recommendation modules
Code Reuse	High	Common functions reused across features
Maintainability	High	Easy to update individual modules
Scalability	High	Supports adding new crops, datasets, and ML models
Extensibility	High	Can integrate fertilizer recommendation, disease detection, and yield prediction
Portability	High	Runs on Windows, Linux, and macOS
Model Reusability	High	Random Forest .pkl model reused without retraining
Overall Assessment	Excellent	Modular, maintainable, reusable, and scalable architecture